Linear Algebra and NumPy

Hoa Vu

Matrix Operations

Vectors and Matrices

A **vector** $x \in \mathbb{R}^n$ is a column of n numbers. A **matrix** $A \in \mathbb{R}^{m \times n}$ has m rows and n columns.

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \qquad A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

- x has shape $(3,)$; A has shape $(2, 3)$.
- Entry A_{ij} is in row i, column j.

Matrix Addition and Scalar Multiplication

For $A, B \in \mathbb{R}^{m \times n}$ and scalar c:

$$(A + B)_{ij} = A_{ij} + B_{ij}, \qquad (cA)_{ij} = c \cdot A_{ij}.$$

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}, \qquad 3 \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}.$$

Both operations are entry-wise; both matrices must have the same shape.

Transpose

The **transpose** $A^\top \in \mathbb{R}^{n \times m}$ swaps rows and columns: $(A^\top)_{ij} = A_{ji}$.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \implies A^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

- $(A^\top)^\top = A$.
- $(AB)^\top = B^\top A^\top$.
- A matrix A is **symmetric** if $A = A^\top$.

Matrix-Vector Multiplication

For $A \in \mathbb{R}^{m \times n}$ and $x \in \mathbb{R}^n$, the product $y = Ax \in \mathbb{R}^m$ has entries:

$$y_i = \sum_{k=1}^{n} A_{ik}\, x_k.$$

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1\cdot 1 + 2\cdot 2 \\ 3\cdot 1 + 4\cdot 2 \\ 5\cdot 1 + 6\cdot 2 \end{bmatrix} = \begin{bmatrix} 5 \\ 11 \\ 17 \end{bmatrix}.$$

Interpretation: Ax is a *linear combination* of the columns of A with coefficients from x.

Matrix-Matrix Multiplication

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the product $C = AB \in \mathbb{R}^{m \times p}$ has entries:

$$C_{ij} = \sum_{k=1}^{n} A_{ik}\, B_{kj}.$$

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1{\cdot}5 + 2{\cdot}7 & 1{\cdot}6 + 2{\cdot}8 \\ 3{\cdot}5 + 4{\cdot}7 & 3{\cdot}6 + 4{\cdot}8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

- The inner dimensions must match: $(m \times n)(n \times p) \to (m \times p)$.
- In general, $AB \neq BA$.

Dot Product

The **dot product** of $u, v \in \mathbb{R}^n$:

$$u \cdot v = u^\top v = \sum_{i=1}^{n} u_i v_i.$$

Example: $u = [1, 2, 3]^\top$, $v = [4, 5, 6]^\top$:

$$u \cdot v = 1{\cdot}4 + 2{\cdot}5 + 3{\cdot}6 = 32.$$

- Geometric interpretation: $u \cdot v = \|u\|\|v\| \cos\theta$.
- $u \cdot v = 0$ means u and v are *orthogonal*.
- Each entry of Ax is a dot product of a row of A with x.

Computational Cost

Counting Operations

We measure cost by the number of *floating-point operations* (flops).

- Addition, subtraction, multiplication, division: each counts as 1 flop.
- We use big-O notation to describe how cost scales with input size.
- Example: summing n numbers costs $O(n)$ flops.

Cost: Vector Addition and Dot Product

Vector addition $u + v$, $u, v \in \mathbb{R}^n$:

- Add entry i to entry i, for $i = 1, \ldots, n$.
- n additions total.
- Cost: $O(n)$.

Both scale *linearly* with the vector length.

Dot product $u^\top v$, $u, v \in \mathbb{R}^n$:

- n multiplications, $n - 1$ additions.
- Cost: $O(n)$.

Cost: Matrix-Vector Multiplication

$y = Ax$, where $A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$, $y \in \mathbb{R}^m$.

$$y_i = \sum_{k=1}^{n} A_{ik}\, x_k \quad \text{for } i = 1, \ldots, m.$$

- Each y_i requires one dot product of length n: $O(n)$ flops.
- There are m such dot products.
- Total cost: $O(mn)$.

Special case: square $n \times n$ matrix costs $O(n^2)$.

Cost: Matrix-Matrix Multiplication

$C = AB$, where $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, $C \in \mathbb{R}^{m \times p}$.

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj} \quad \text{for } i = 1, \ldots, m,\ j = 1, \ldots, p.$$

- mp entries in C; each requires $O(n)$ flops.
- Total cost: $O(mnp)$.
- Square case ($m = n = p$): $O(n^3)$.

Why $O(n^3)$ Matters

Multiplying two $n \times n$ matrices: n^2 entries, each requiring n multiplications and $n-1$ additions $\approx 2n^3$ flops.

n	n^2	n^3	Time at 10^9 flops/s
100	10^4	10^6	< 1 ms
1 000	10^6	10^9	~ 1 s
10 000	10^8	10^{12}	~ 17 min

Python Loop vs. Vectorized

Loop (Python)

- Each iteration: type check, object lookup, function call overhead.
- Roughly 100$\times$ slower than C.

Vectorized (NumPy)

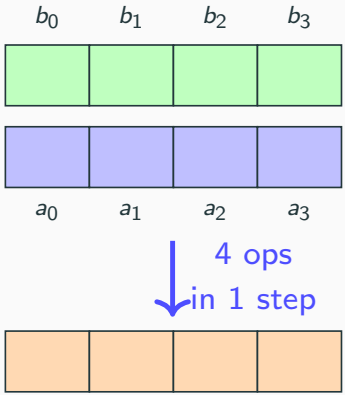
- Single call dispatches to a compiled C routine (BLAS).
- Data stored as a contiguous block of `float64`.
- Modern CPUs process multiple elements per clock (SIMD).

SIMD: One Instruction, Multiple Data

A modern CPU can add **4 numbers at once** using a single instruction (SIMD register holds 4× `float64`).

NumPy feeds data directly into these registers. A Python loop cannot — each step has overhead before any arithmetic.

SIMD (NumPy)



NumPy

Creating Arrays

```
import numpy as np

a = np.array([1, 2, 3])           # shape (3,)
A = np.array([[1, 2], [3, 4]])    # shape (2, 2)

np.zeros((3, 4))      # 3 by 4 matrix of zeros
np.ones((2, 3))       # 2 by 3 matrix of ones
np.eye(4)             # 4 by 4 identity matrix
np.arange(0, 10, 2)   # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5) # [0., 0.25, 0.5, 0.75, 1.]
```

Element-wise Operations and Broadcasting

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b          # [5, 7, 9]        element-wise add
a * b          # [4, 10, 18]      element-wise multiply
a ** 2         # [1, 4, 9]
a + 10         # [11, 12, 13]     scalar broadcast

A = np.array([[1, 2], [3, 4]])
A + np.array([10, 20])  # adds [10,20] to every row
```

Note: `a * b` is element-wise, not a dot product.

Matrix Operations in NumPy

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
x = np.array([1, 2])

A.T           # transpose:         [[1,3],[2,4]]
A @ B         # matrix multiply:   [[19,22],[43,50]]
A @ x         # matrix-vector:     [5, 11]
x @ x         # dot product:       5
```

Key: `A * B` is entry-wise; `A @ B` is matrix multiplication.

Matrix Multiply: NumPy vs. Python Loop

```
C = A @ B   # NumPy

C = np.zeros((n, n))
for i in range(n):
  for j in range(n):
    for k in range(n):
      C[i,j] += A[i,k] * B[k,j]
```

Benchmark: $n = 500$

Method	Time
NumPy (`@`)	0.032 s
Python loop	28.15 s

$\approx 885\times$ slower

Useful Reductions

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

A.sum()          # 21        all entries
A.sum(axis=0)    # [5,7,9]   sum each column
A.sum(axis=1)    # [6,15]    sum each row
A.mean()         # 3.5
A.max()          # 6
A.argmax()       # 5   flat index of maximum

np.linalg.norm(A[0])    # L2 norm of first row
```

Solve a system of linear equations

A System of Linear Equations

Find x_1, x_2, x_3 such that:

$$\begin{cases} 2x_1 + x_2 - x_3 = 8 \\ -3x_1 - x_2 + 2x_3 = -11 \\ -2x_1 + x_2 + 2x_3 = -3 \end{cases}$$

In matrix form $Ax = b$:

$$\underbrace{\begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}}_{A} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \underbrace{\begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix}}_{b}$$

Solution: $x_1 = 2,\ x_2 = 3,\ x_3 = -1$.

Solving $Ax = b$ with NumPy

Use `np.linalg.solve` — not `np.linalg.inv`.

```
import numpy as np

A = np.array([[ 2,  1, -1],
              [-3, -1,  2],
              [-2,  1,  2]], dtype=float)
b = np.array([8, -11, -3], dtype=float)

x = np.linalg.solve(A, b)
print(x)       # [ 2.  3. -1.]
print(A @ x)   # [ 8. -11.  -3.]  -- verify
```

When Does $Ax = b$ Have No Unique Solution?

No solution

Equations are contradictory.

$$\begin{cases} x + y = 2 \\ x + y = 3 \end{cases}$$

A is singular; the two lines are parallel and never meet.

Infinitely many solutions

One equation is redundant.

$$\begin{cases} x + y = 2 \\ 2x + 2y = 4 \end{cases}$$

A is singular; the two equations describe the same line.

In both cases $\det(A) = 0$ and `np.linalg.solve` raises `LinAlgError: Singular matrix`.

Least Square Regression

The Big Idea

You have points that *almost* lie on a line.
No single line passes through all of them.

Which line fits **best**?

Measuring the Error

For a candidate line $y = ax + b$, the **residual** at point i is the vertical gap between the data and the line:

$$r_i = y_i - (ax_i + b)$$

- $r_i = 0$ means the line passes exactly through point i.
- r_i large means the line is far off at that point.

Minimise the Total Squared Error

Least squares picks a and b to minimise the sum of squared residuals (recall MSE):

$$\min_{a,\,b} \sum_{i=1}^{n} \big(y_i - a x_i - b\big)^2$$

Writing It as a Matrix Problem

Stack all n data points into one system $Ax = b$:

$$\underbrace{\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}}_{A} \begin{bmatrix} a \\ b \end{bmatrix} = \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}}_{b}$$

With $n > 2$ this has **no exact solution** —
more equations than unknowns.

The Normal Equations

The least-squares solution satisfies the **normal equations**:

$$A^\top A\, x = A^\top b$$

Solving gives:

$$x = (A^\top A)^{-1} A^\top b$$

- $A^\top A$ is a small 2×2 matrix — easy to invert.
- NumPy's `lstsq` does this for you, stably and efficiently.

Multiple Variables

The same framework extends to **multiple predictors** $x_1, x_2, \ldots, x_p$:

$$y = a_1 x_1 + a_2 x_2 + \cdots + a_p x_p + b$$

Each data point gives one row in A:

$$A = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_p^{(1)} & 1 \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_p^{(2)} & 1 \\ \vdots & \vdots & & \vdots & \vdots \\ x_1^{(n)} & x_2^{(n)} & \cdots & x_p^{(n)} & 1 \end{bmatrix}$$

The normal equations $A^\top A\, x = A^\top b$ are identical — only the size of A changes.

Example: predicting house price from size, number of rooms, age.

A Dataset: 10 Noisy Points

We have 10 measurements roughly following $y = 2x + 1$:

x	1	2	3	4	5	6	7	8	9	10
y	1.8	6.0	6.2	10.5	9.8	15.1	13.5	18.9	17.5	23.2

Goal: find the line $y = ax + b$ that **best fits** all 10 points.

This is an overdetermined system (10 equations, 2 unknowns) — minimise $\|Ax - b\|^2$.

Fitting the Line with `lstsq`

```python
import numpy as np

x = np.array([1,2,3,4,5,6,7,8,9,10], dtype=float)
y = np.array([1.8,6.0,6.2,10.5,9.8,15.1,13.5,18.9,17.5,23.2])

# Design matrix: each row is [x_i, 1]
A = np.column_stack([x, np.ones_like(x)])

coeffs, _, _, _ = np.linalg.lstsq(A, y, rcond=None)
a, b = coeffs
print(f"slope={a:.4f}, intercept={b:.4f}")
# slope=2.1267, intercept=0.5533
```

The noise shifts the fit slightly from the true $y = 2x + 1$, but the line still captures the trend.

Fitted Line